VarSplat: Uncertainty-aware 3D Gaussian Splatting for Robust RGB-D SLAM

Anh Thuan Tran, Jana Košecká
Department of Computer Science
George Mason University
{atran92, kosecka}@gmu.edu

https://anhthuan1999.github.io/varsplat/

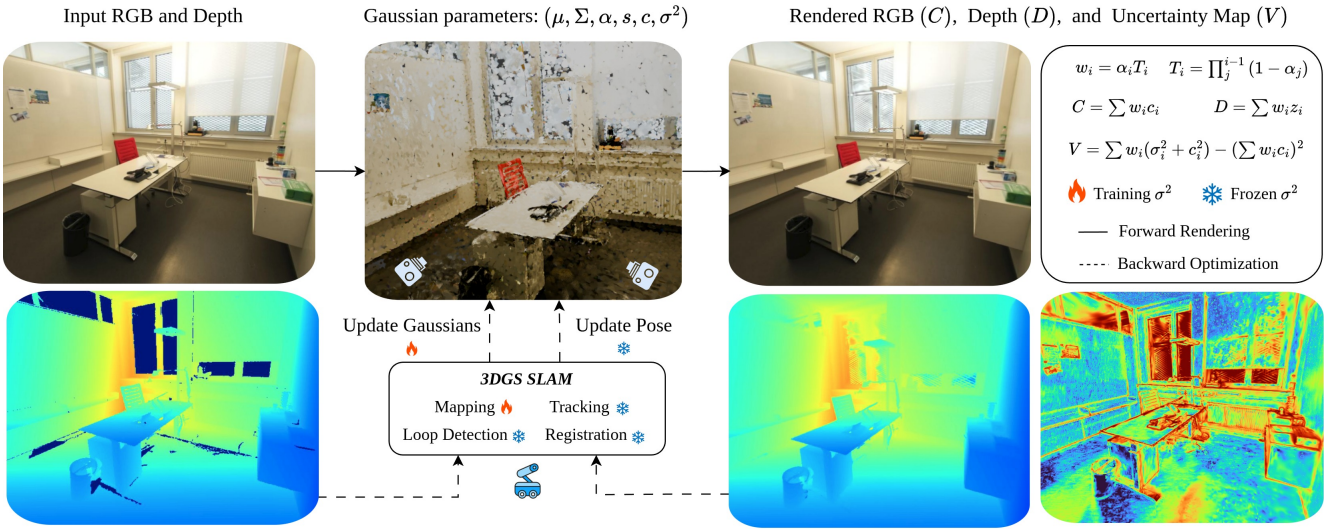

Figure 1. **VarSplat.** Given RGB-D inputs, each 3D Gaussian jointly learns position, orientation, scale, color, opacity, and appearance variance σ^2. During mapping, σ^2 is optimized jointly with other Gaussian parameters. The rendered per-pixel uncertainty V serves as a confidence weight during tracking and registration, while the per-Gaussian variance σ^2 further guides loop detection, enabling robust and uncertainty-aware SLAM.

Abstract

Simultaneous Localization and Mapping (SLAM) with 3D Gaussian Splatting (3DGS) enables fast, differentiable rendering and high-fidelity reconstruction across diverse real-world scenes. However, existing 3DGS-SLAM approaches handle measurement reliability implicitly, making pose estimation and global alignment susceptible to drift in low-texture regions, transparent surfaces, or areas with complex reflectance properties. To this end, we introduce VarSplat, an uncertainty-aware 3DGS-SLAM system that explicitly learns per-splat appearance variance. By using the law of total variance with alpha compositing, we then render differentiable per-pixel uncertainty map via efficient, single-pass rasterization. This map guides tracking, submap registration, and loop detection toward focusing on reliable regions and contributes to more stable optimization. Experimental results on Replica (synthetic) and TUM-RGBD, ScanNet, and ScanNet++ (real-world) show that VarSplat improves robustness and achieves competitive or superior tracking, mapping, and novel view synthesis rendering compared to existing studies for dense RGB-D SLAM.

1. Introduction

Simultaneous Localization and Mapping (SLAM) [18] is significant for robotics and AR/VR applications by giving the autonomous systems ability to actively explore unknown environments while incrementally building a map. Classical methods [26, 27, 44] based on voxels, TSDFs, surfels, or meshes can achieve real-time tracking and mapping but struggle to capture photorealistic appearance. To improve visual fidelity, Neural radiance fields (NeRF) [25]

have been integrated into SLAM systems [39, 55], though implicit volumetric rendering remains costly for real-time performance. 3D Gaussian Splatting (3DGS) [17] further addresses this bottleneck by rasterizing anisotropic Gaussians with alpha compositing, enabling fast, differentiable rendering. Recent 3DGS-SLAM systems [16, 48, 51, 54] adopt this explicit representation and jointly optimize camera poses and Gaussian parameters.

Despite these advances, a key limitation exists: *measurement reliability is rarely modeled explicitly*. Uniform photometric weighting leaves pose estimation vulnerable in low-texture regions, around depth discontinuities, and on reflective surfaces. For safety-critical systems, explicitly quantifying uncertainty is important, providing confidence alongside rendering quality. Prior efforts address uncertainty primarily on the geometric side (e.g., depth variance or probabilistic filters) [11, 31, 42] or rely on pretrained predictors [53]. The appearance uncertainty that directly reflects instability in 3DGS rendering has not been treated as a first-class quantity in online dense SLAM.

To address these challenges, we introduce **VarSplat**, an uncertainty-aware RGB-D SLAM system leveraging 3D Gaussian Splatting. VarSplat integrates explicit uncertainty into the map representation by learning per-splat appearance variance σ^2 that is trained along with standard Spherical Harmonics (SH) coeffcients. Using the law of total variance, we derive propagation through 3DGS rasterizer to obtain differentiable per-pixel uncertainty map V. The system is trained end-to-end online, jointly optimizing poses, Gaussian parameters, and σ^2 as the map grows. For downstream tasks, uncertainty is applied across all three stages. In *tracking*, the per-pixel uncertainty map V provides short-horizon reliability measure within each submap, improving frame-to-frame pose updates. In *registration*, the same per-pixel reliability supports mid-range alignment between overlapping submaps. For *loop detection*, per-splat variances σ^2 modulate submap similarity and correct long-range drift. To the best of our knowledge, VarSplat is the first 3DGS–SLAM system to learn per-splat appearance variance and render per-pixel uncertainty in online setting. In summary, our contributions can be shown as follows:

- We introduce *VarSplat*, an RGB-D 3DGS–SLAM system that learns per-splat appearance variance σ^2 to render differentiable per-pixel uncertainty map V while maintaining single-pass rasterization efficiency.
- We integrate uncertainty at both representation and renderer levels: poses, Gaussian parameters, and variance σ^2 are optimized together in a fully online, end-to-end submap pipeline.
- Extensive experiments on Replica, TUM-RGBD, ScanNet, ScanNet++ show improved robustness and competitive or superior tracking, reconstruction, and novel view synthesis rendering against 3DGS-SLAM baselines.

2. Related Work

In this section, we briefly go through various approaches to dense SLAM that leverage radiance fields for 3D representations and uncertainty quantification.

Dense Visual SLAM. Traditional dense SLAM, with DTAM [27] and KinectFusion [26], achieved real-time tracking and mapping from RGB-D images using explicit volumetric or surface representations such as TSDFs [6, 43], voxels [13, 49] and surfels [33, 44]. While effective for real-time pose estimation, these approaches struggle to provide texture-rich, high-fidelity reconstructions and synthesize novel views. iMap [39] integrated Neural Radiance Fields (NeRF) [25] into tracking and mapping, and Nice-SLAM [55] improved scalability with multi-resolution feature grids. Additional efficiency representations and global consistency concerns were addressed in [15, 23, 30, 41, 46, 52]. Recently, SplaTAM [16] used 3D Gaussians [17] for dense RGB-D SLAM with per-pixel photometric losses, and submap-based systems [24, 48, 51] balanced accuracy and efficiency through incremental mapping. VarSplat follows this general 3DGS-SLAM architecture but treats measurement reliability explicitly.

Uncertainty Quantification for Radiance Fields. In Neural Radiance Fields, Stochastic NeRF [34] and Conditional-Flow NeRF [35] attempted to quantify uncertainty by reparametrizing NeRF with Bayesian model. However, these approaches focused on predictive uncertainty and do not capture parameter uncertainty grounded in observed information. Bayes' Rays [9] introduced spatially parameterized perturbation field and interpolates uncertainty at NeRF query points, estimating it with Laplace approximation. Information-driven studies used uncertainty for decision making. FisherRF [14] modeled uncertainty with Fisher information for active view selection. Bayesian NeRF [21] modeled explicitly quantifies uncertainty in the geometric volume structure, particularly volume density and occupancy. ActiveNeRF [7] rendered per-pixel variance map using the neural network to guide model acquisition. Recent methods [1, 22] integrated stochasticity into 3D Gaussian Splatting [17] pipeline through variational inference and Monte Carlo sampling to approximate pixel-level variance. A probabilistic view also treats 3DGS optimization as SGLD sampling [19]. Sensitivity-based [10] approach related uncertainty to model sensitivity for pruning. Semantic 3DGS [45] estimates per-pixel variance while rasterizing semantic maps. VarSplat introduces a conceptually different method that learns per-splat appearance variance and leverage Gaussian rasterizer to render per-pixel uncertainty map. This approach allows us to maintain single-pass rasterization efficiency in online setting.

Uncertainty in SLAM. Incorporating uncertainty in SLAM is important for robust localization and consistent maps under real-time constraints. REMODE [29] models

per-pixel depth distributions, then uses their variance into mapping, while SVO [8] provides depth filters with explicit uncertainty updates. DeepFactors [4] presents a real-time probabilistic framework for dense monocular SLAM. Within neural SLAM, UncLe-SLAM [31] learns per-pixel depth uncertainty and reweights tracking and mapping. Uni-SLAM [42] defines predictive uncertainty for neural implicit fields and uses it to reweight pixel losses. CG-SLAM [11] builds 3D Gaussian field and models depth uncertainty to guide optimization and select informative Gaussians. For dynamic scenes, WildGS-SLAM [53] predicts an uncertainty map from DINOv2 features to filter moving distractors, and GLC-SLAM [47] uses uncertainty for keyframe selection to stabilize submap optimization. Rather than modeling geometric depth variance [11, 31] or relying on pretrained predictors [53], we learn per-splat appearance variance and propagate it through alpha compositing and the law of total variance to obtain a differentiable per-pixel uncertainty map. In contrast to variance from ray-termination probabilities [42], our variance is learned from the rasterizer and updated frame by frame.

3. Method

VarSplat is an RGB-D SLAM approach that jointly estimates camera poses and incrementally updates 3D Gaussian Splatting (3DGS) map from input frames, following the general pipeline of [51, 54]. However, pose estimation through photometric optimization can suffer from unreliable observations in low-texture regions, reflective surfaces, and areas near depth discontinuities, which can destabilize this process and potential drift. To address these issues, we introduce a novel uncertainty quantification pipeline based on per-pixel uncertainty map rendering. Specifically, differentiable uncertainty map V is rendered through 3DGS rasterizer via the law of total variance and alpha compositing using per-splat variance appearance σ^2. Unlike previous methods relying on pretrained predictors [20, 53], VarSplat learns this variance parameter from scratch, jointly optimizing them with appearance and geometry during mapping. The resulting σ^2 and V are used for tracking in local submaps, registration between them, and loop detection to improve robustness and consistency in long runs. An overview of the architecture is shown in Fig. 2.

3.1. Per-pixel uncertainty rendering

Intuition. Before going deeper in 3D Gaussian Splatting, we first clarify the role of our per-splat appearance variance σ^2, which is different from spatial covariance Σ defining its geometric extent and Spherical Harmonics (SH) coefficients defining mean appearance. In contrast, σ^2 models uncertainty around that mean color. Even when the SH color correctly models the mean, view-dependent appearance, the actual color observations at a splat can vary across viewpoints. As illustrated in Fig. 1, larger variances appear near depth discontinuities or occlusion boundaries and in reflective or transparent regions. This is because small view changes can alter visibility and alpha weights of overlapping splats at different depths, leading to inconsistent color observations. Moreover, we denote the appearance variance as σ^2 to enforce positivity and follow the conventional notation of Gaussian-form uncertainty in the appearance domain. Using σ^2 makes it clear that we optimize a true statistical variance rather than a free scale weight. During rendering, these variances are composited into per-pixel variance V, allowing the renderer to quantify uncertainty directly from the splatting process.

3D Gaussian Splatting. Followed by [23, 51, 54], we represent the scene as a set of submaps P^s, each containing N^s 3D Gaussians [17]. Each Gaussian G_i is defined by its mean position $\mu_i \in \mathbb{R}^3$, opacity $\alpha_i \in \mathbb{R}$, scales $s_i \in \mathbb{R}^3$ and covariance matrix $\Sigma_i \in \mathbb{R}^{3\times3}$. The corresponding color $c_i \in \mathbb{R}^3$ is derived from spherical harmonics. We further include an additional parameter $\sigma_i^2 \in \mathbb{R}^3$ to represent the appearance variance. In summary, each submap can be described as:

$$P^s = \{G_i^s(\mu_i, \Sigma_i, \alpha_i, s_i, c_i, \sigma_i^2) | i = 1, \dots, N^s\} \quad (1)$$

With standard alpha compositing [17, 51, 56], the transmittance and weights are:

$$w_i = T_i \alpha_i, \quad T_i = \prod_j^{i-1} (1 - \alpha_j) \quad (2)$$

and the rendered color C and depth D are:

$$C = \sum_i w_i c_i, \quad D = \sum_i w_i z_i \quad (3)$$

where $c_i \in \mathbb{R}^3$ is the mean color and $z_i \in \mathbb{R}$ is the camera-space depth of the projected mean of G_i.

Variance rendering. For a random variable X and conditioning variable Z, the law of total variance states:

$$\mathrm{Var}[X] = \mathbb{E}\big[\mathrm{Var}[X|Z]\big] + \mathrm{Var}\big(\mathbb{E}[X|Z]\big). \quad (4)$$

In our setting, X denotes pixel color value, and conditioning variable Z is 3D Gaussians. The per-pixel uncertainty $\mathrm{Var}[X]$ can be then decomposed into the expected per-splat variance $\mathbb{E}\big[\mathrm{Var}[X|Z]\big]$ and the variance of the splats $\mathrm{Var}\big(\mathbb{E}[X|Z]\big)$. Conditioned on splat i-th, we have splat color c_i and variance σ_i^2 can be described as:

$$\mathbb{E}[X|Z=i] = c_i, \quad \mathrm{Var}[X|Z=i] = \sigma_i^2. \quad (5)$$

Therefore, we can obtain expected per-splat variance and color by alpha blending, similar to how we compute per-pixel color in Eq. (3):

$$\mathbb{E}[X|Z] = \sum_i w_i c_i, \quad \mathbb{E}\big[\mathrm{Var}[X|Z]\big] = \sum_i w_i \sigma_i^2 \quad (6)$$

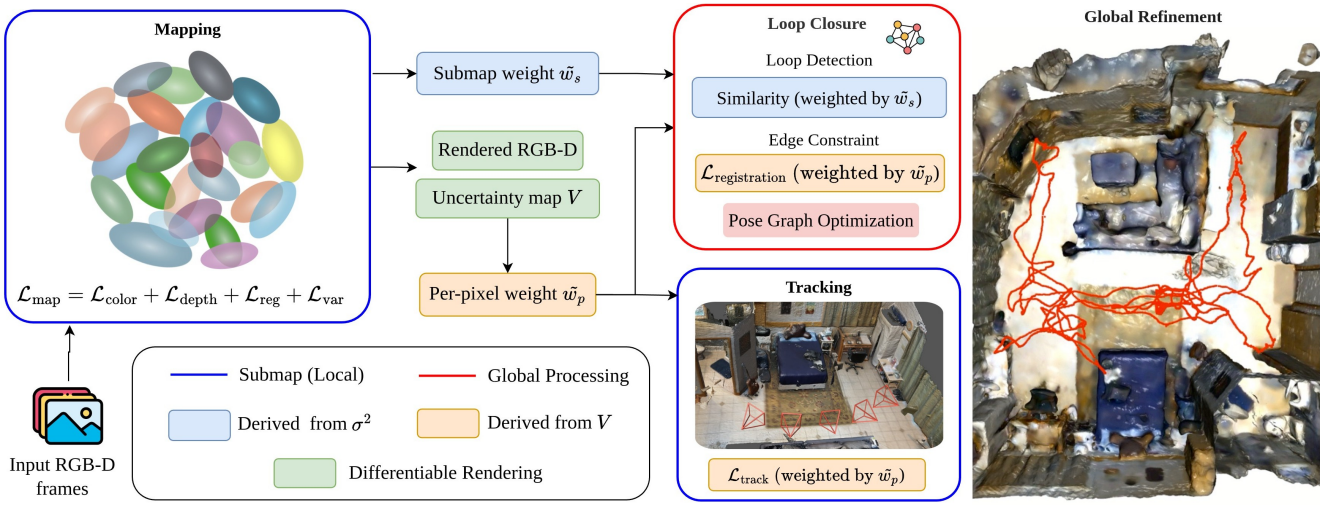

Figure 2. VarSplat architecture. During mapping, each 3D Gaussian jointly learns position, appearance, and variance σ^2. The per-splat variances are composited into per-pixel uncertainty V through the differentiable rasterizer in mapping. This uncertainty map is then served as confidence in tracking and registration, while σ^2 is used for uncertainty-aware loop detection.

The variance of splats $\text{Var}\big(\mathbb{E}[X|Z]\big)$ is then computed using second moment of a distribution and Eq. (6):

$$\text{Var}(\mathbb{E}[X|Z]) = \mathbb{E}[(X|Z)^2] - (\mathbb{E}[X|Z])^2 \tag{7}$$

$$\text{Var}(\mathbb{E}[X|Z]) = \sum_i w_i c_i^2 - \Big(\sum_i w_i c_i\Big)^2 \tag{8}$$

From Eqs (4), (6) and (8), we can compute per-pixel variance $\text{Var}[X]$ denoted as V using per-splat color c_i and variance σ_i^2.

$$V = \sum_i w_i(\sigma_i^2 + c_i^2) - \Big(\sum_i w_i c_i\Big)^2 \tag{9}$$

We use V to optimize σ_i^2 during mapping and as per-pixel weights for tracking and submap registration. By sharing the same single-pass rasterization as color and depth, V enables efficient, online, in-the-loop reliability estimation.

3.2. Mapping

Following [51, 54], VarSplat jointly estimates camera poses $T_j \in SE(3)$ and Gaussian submap parameters $G_i(\mu_i, \Sigma_i, \alpha_i, s_i, c_i, \sigma_i^2)$ from streaming RGB-D input. Given the current estimate T_j and keyframe color I_j, depth D_j images, we differentiably render the corresponding view from the current submap to obtain the predicted color $\hat{I}_j$, predicted depth $\hat{D}_j$, and per-pixel uncertainty map V_j using the compositing equations Eqs (3) and (9).

Submap Optimization. Each submap P^s covers a limited spatial region to preserve local consistency and incremental map growth. Following [51, 54], a new submap is initialized when the camera moves beyond a spatial threshold from the current submap centroid or when accumulated tracking uncertainty passes a preset limit. The first keyframe seeds Gaussians by backprojecting depth points, and subsequent frames expand coverage by adding Gaussians in unobserved regions or merging overlapping ones. After sufficient observations, submap Gaussian parameters are jointly optimized to better align appearance and geometry with their associated keyframes. We leverage this stage to train the per-splat appearance variance with the following loss:

$$\mathcal{L}_{\text{map}} = \lambda_{\text{color}} \cdot \mathcal{L}_{\text{color}} + \lambda_{\text{depth}} \cdot \mathcal{L}_{\text{depth}} + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}} + \lambda_{\text{var}} \cdot \mathcal{L}_{\text{var}} \tag{10}$$

where λ_* are hyperparameters. For color supervision, we use a weighted combination of L1 and SSIM [17], while depth loss is L1 between rendered and ground-truth depth. Similar to [48], we also add regularization loss to control Gaussian scales:

$$\mathcal{L}_{\text{color}} = (1-\lambda_{\text{SSIM}})\|\hat{I}-I\|_1 + \lambda_{\text{SSIM}}(1-\text{SSIM}(\hat{I}, I)) \tag{11}$$

$$\mathcal{L}_{\text{depth}} = \|\hat{D} - D\|_1, \quad \mathcal{L}_{\text{reg}} = \|\hat{s} - s\|_1 \tag{12}$$

Learning Variance from Scratch. Motivated by likelihood perspective similar to ActiveNeRF [7], which treats rendered rays as independent distributions, we learn per-splat variance with same Gaussian negative log-likelihood. Unlike ActiveNeRF, we optimize this variance directly in 3DGS rasterizer without relying on neural network. As shown in Eq. (9), the per-pixel variance V is composited from both the squared mean color c^2 and the per-splat variance σ^2. To stay consistent with the Gaussian view, we use square L2 (MSE) for variance loss $\mathcal{L}_{\text{var}}$. This design reflects the Gaussian negative log-likelihood, where variance represents the second-order moment of residuals. Using L1

(which models Laplace scale) would break this relationship, leading to inconsistent variance estimation.

$$\mathcal{L}_{\text{var}} = \frac{1}{2V}\left(\|\hat{I} - I\|_2^2 + \|\hat{D} - D\|_2^2\right) + \log(V) \quad (13)$$

Unlike previous works that rely on pretrained uncertainty predictors [20, 53], VarSplat learns the per-splat appearance variance σ_i^2 end-to-end as a differentiable parameter of each Gaussian. Gradients from both color and depth residuals update per-splat variance, so the predicted variance reflects measurement reliability and avoids overconfidence in reflective, transparent, or glossy surfaces. The derivative with respect to the predicted per-pixel variance is given by:

$$\frac{\partial \mathcal{L}_{\text{var}}}{\partial V} = -\frac{\|\hat{I} - I\|_2^2 + \|\hat{D} - D\|_2^2}{2V^2} + \frac{1}{V} \quad (14)$$

which, by the chain rule and Eq.(9), gives per-splat gradient:

$$\frac{\partial \mathcal{L}_{\text{var}}}{\partial \sigma_i^2} = \frac{\partial \mathcal{L}_{\text{var}}}{\partial V}\frac{\partial V}{\partial \sigma_i^2} = \frac{\partial \mathcal{L}_{\text{var}}}{\partial V}\, w_i. \quad (15)$$

This allows per-splat variance σ_i^2 to adapt dynamically to residual magnitudes, while per-pixel uncertainty V naturally captures fluctuations in opacity and visibility. Therefore, the system can effectively observe high uncertainty at depth discontinuities and occlusion boundaries where transmittance weights w_i shift abruptly. Through optimization of camera poses, Gaussian parameters, and appearance variance, each submap progressively evolves into a locally consistent, uncertainty-aware representation.

3.3. Downstream Pose Estimation

Weighting. Given the per-splat and per-pixel variances, we leverage them as confidence measures at both the pixel and submap levels to reduce the impact of unstable regions and photometric noise during pose estimation. Thanks to the explicit representation and the parallel optimization of poses and Gaussian parameters, we can simultaneously freeze the variance during tracking and registration, while still training it end-to-end through the mapping stage without interrupting the gradient flow. To obtain effective confidence weights, we normalize variance across pixels within a frame and across splats within a submap using a median-centered log scaling:

$$\begin{aligned} \widetilde{V} &= \operatorname*{median}_{\Omega}(\log(V)), \quad \widetilde{w}_p = \exp[-(\log V - \widetilde{V})/\tau], \\ \widetilde{\sigma^2} &= \operatorname{median}(\log \sigma^2), \quad \widetilde{w}_s = \exp[-(\log \sigma^2 - \widetilde{\sigma^2})/\tau] \end{aligned} \quad (16)$$

where $\tau > 0$ controls the sharpness of the uncertainty weighting and Ω denotes the valid pixels after applying inlier and alpha masks. The normalized weights $\widetilde{w_p}$ and $\widetilde{w_s}$ are used in tracking and loop closure modules. Therefore, pixels or splats with larger-than-median variance receive smaller weights, while more reliable regions receive stronger supervision.

Tracking. Given an incoming frame (I, D), the tracker estimates the current camera pose T_j with respect to the active submap. We also apply both alpha, inlier mask to stabilize tracking and denote Ω as valid pixels after masking. Unlike depth maps, RGB images are more susceptible to viewpoint changes, low texture, and occlusions. Therefore, optimizing a pure photometric loss for pose refinement can lead to unstable gradients. To address this, we leverage the rendered per-pixel variance as an uncertainty-aware weight to adaptively constrain unreliable pixels.

$$\mathcal{L}_{\text{track}} = \sum \lambda_c \left(\widetilde{w_p} \odot \|\hat{I} - I\|_1\right) + (1 - \lambda_c)\|\hat{D} - D\|_1 \quad (17)$$

where $0 \le \lambda_c \le 1$ balances the contribution between photometric and geometric losses, and $\widetilde{w_p}$ is the normalized variance weight defined earlier. During tracking we freeze variance parameters and stop gradients through $\widetilde{w_p}$.

Loop Detection. Following LoopSplat [54], each submap is represented by keyframe descriptors for loop detection. Instead of computing per-pixel confidence for each keyframe, we use the learned per-splat variance σ_i^2 to derive a submap-level reliability weight that modulates similarity. At the submap level, rather than explicitly down-weighting appearance features, we compute the ratio of opacity α before and after applying the variance-based weights. This ratio encodes how much reliable appearance information remains in the submap without the need to penalize each submap separately.

$$r = \frac{\sum_j \widetilde{w_s}\alpha_j}{\sum_j \alpha_j}, \quad \text{sim} = \text{cross\_sim} \odot (r_q * r_{db}) \quad (18)$$

where r_{q} denotes the opacity ratio of the query submap, and r_{db} denotes the opacity ratios of the database submaps used for computing similarity scores.

Registration. After detecting a loop, we register the matched submaps by localizing the query keyframes in the database submap, following [54]. However, instead of using the native rendering loss [51], we apply the variance-derived per-pixel weight $\widetilde{w}_p$ to modulate photometric loss, stabilizing the estimated viewpoint transformation. Formally, the registration loss is defined as:

$$\mathcal{L}_{\text{registration}} = \sum \widetilde{w_p} \odot \|\hat{I} - I\|_1 + \|\hat{D} - D\|_1 \quad (19)$$

Merging Submaps and Global Refinement. We merge all submaps into a consistent global representation with TSDF fusion. The merged geometry is then used to initialize Gaussian centers in the global map, which are subsequently refined using the color reconstruction loss $\mathcal{L}_{\text{color}}$ as in [17]. In this final stage, we exclude uncertainty weights to focus on appearance quality, since unstable regions are already controlled during tracking, loop detection, and registration.

4. Experiments

In this section, we evaluate VarSplat against existing baselines on both synthetic and real-world datasets. Moreover, we conduct ablation studies of the proposed uncertainty model, measuring its impact on tracking, registration, and loop detection. Implementation details and additional results are provided in the Supplementary Material.

4.1. Experimental Setup

Datasets. We evaluate VarSplat on four datasets: Replica [36], TUM-RGBD [38], ScanNet [5], and ScanNet++ [50]. Following prior works [16, 30, 51, 54], we use the same eight sequences in Replica. On TUM-RGBD, we evaluate five sequences used in [54]. For fair comparison on ScanNet with common baselines [16, 48, 51], we report results on six scenes. Following [48, 54], we evaluate the same five scenes a-e with IDs b20a261fdf, 8b5caf3398, fb05e13ad1, 2e74812d00, and 281bc17764 on ScanNet++.

Baselines. For camera pose estimation tracking, we use the average root mean square absolute trajectory error (ATE RMSE) [37] on keyframes. For reconstruction quality, we follow [30, 54] to evaluate meshes with voxel size of 1cm. We compute L1 on rendered depth and the F1 score against ground truth mesh vertices as in [54, 55]. For rendering quality, we evaluate input training views, reporting PSNR, SSIM, and LPIPS. On ScanNet++, we also report novel view synthesis on the five standard scenes for fair comparison with [51, 54]. In the tables, we highlight results, with **best**, second, and third.

Baselines. We compare VarSplat with state-of-the-art methods for dense radiance-field-based SLAM. In NeRF, we evaluate GO-SLAM [52], NICE-SLAM [55], Point-SLAM [30], ESLAM [15], Loopy-SLAM [23], and Uni-SLAM [42]. For 3D Gaussian Splatting, we evaluate MonoGS [24], SplaTAM [16], Gaussian-SLAM [51], LoopSplat [54], and CG-SLAM [11]. Among these, Uni-SLAM models uncertainty via termination-probability field, and CG-SLAM incorporates depth-driven geometric uncertainty.

4.2. Quantitative Evaluation

Tracking. We report camera tracking results in Tables 1 to 4. On Replica, VarSplat improves over existing studies around 10% through high quality synthetic data limits uncertainty's impact through high quality data. On ScanNet++, VarSplat improves ATE RMSE by about 18% over the second best method and ensures robustness in long sequences where others like SplaTAM fail (443.10 cm). Despite lower image quality and incomplete depth on TUM-RGBD, VarSplat stabilizes motion in textureless or reflective regions by guiding correspondences without manual mask. On ScanNet, VarSplat consistently achieves best performance against both neural implicit and 3DGS baselines.

Table 1. Tracking Performance on Replica [36] (ATE RMSE ↓ [cm]). UC indicates uncertainty. The results are highlighted as **best**, second and third. VarSplat achieves the best average performance while remaining competitive across scenes. *Photo-SLAM [12] use ORB-SLAM3 features [3] for tracking and loop closure.

Method	UC	R0	R1	R2	Off0	Off1	Off2	Off3	Off4	Avg.
Neural Implicit Fields										
NICE-SLAM [55]	✗	0.97	1.31	1.07	0.88	1.00	1.06	1.10	1.13	1.06
ESLAM [15]	✗	0.71	0.70	0.52	0.57	0.55	0.58	0.72	0.63	0.63
Point-SLAM [30]	✗	0.61	0.41	0.37	0.38	0.48	0.54	0.69	0.72	0.52
GO-SLAM [52]	✗	0.34	0.29	0.29	0.32	0.30	0.39	0.39	0.46	0.35
Loopy-SLAM [23]	✗	0.24	0.24	0.28	0.26	0.40	0.29	0.22	0.35	0.29
Uni-SLAM [42]	✓	0.49	0.48	0.40	0.37	0.36	0.48	0.56	0.44	0.45
3D Gaussian Splatting										
SplaTAM [16]	✗	0.31	0.40	0.29	0.47	0.27	0.29	0.32	0.72	0.38
MonoGS [24]	✗	0.33	**0.22**	0.29	0.36	0.19	**0.25**	**0.12**	0.81	0.32
Gaussian-SLAM [51]	✗	0.29	0.29	0.22	0.37	0.23	0.41	0.30	0.35	0.31
*Photo-SLAM [12]	✗	0.54	0.39	0.31	0.52	0.44	1.28	0.78	0.58	0.60
LoopSplat [54]	✗	0.28	**0.22**	**0.17**	**0.22**	0.16	0.49	0.20	0.30	0.26
CG-SLAM [11]	✓	0.29	0.27	0.25	0.33	**0.14**	0.28	0.31	**0.29**	0.27
VarSplat (Ours)	✓	**0.20**	0.23	0.19	0.26	0.15	0.34	0.15	0.35	**0.23**

Table 2. Tracking Performance on ScanNet++ (ATE RMSE ↓ [cm]). VarSplat achieves the highest accuracy with robustness on large motion camera.

Method	a	b	c	d	e	Avg.
Neural Implicit Fields						
Point-SLAM [30]	246.16	632.99	830.79	271.42	574.86	511.24
ESLAM [15]	25.15	2.15	27.02	20.89	35.47	22.14
GO-SLAM [52]	176.28	145.45	38.74	85.48	106.47	110.49
Loopy-SLAM [23]	N/A	N/A	25.16	234.25	81.48	113.63
3D Gaussian Splatting						
SplaTAM [16]	1.50	**0.57**	**0.31**	443.10	1.58	89.41
MonoGS [24]	7.00	3.66	6.37	3.28	44.09	12.88
Gaussian-SLAM [51]	1.37	5.97	2.70	2.35	1.02	2.68
LoopSplat [54]	1.14	3.16	3.16	**1.68**	0.91	2.05
VarSplat (Ours)	**1.09**	2.24	2.39	1.90	**0.84**	**1.69**

Table 3. Tracking Performance on TUM-RGBD (ATE RMSE ↓ [cm]). UC indicates uncertainty. VarSplat outperforms both 3DGS and NeRF baselines. *Photo-SLAM [12] use ORB-SLAM3 features [3] for tracking and loop closure.

Method	UC	fr1/desk	fr1/desk2	fr1/room	fr2/xyz	fr3/off.	Avg.
Neural Implicit Fields							
NICE-SLAM [55]	✗	4.26	4.99	34.49	6.19	3.87	10.76
MIPS-Fusion [40]	✗	3.00	N/A	N/A	1.4	4.6	N/A
Point-SLAM [30]	✗	4.34	4.54	30.92	1.31	4.8	8.92
ESLAM [15]	✗	2.47	3.69	29.73	1.11	2.42	7.89
Co-SLAM [41]	✗	2.40	N/A	N/A	1.70	2.40	N/A
GO-SLAM [52]	✗	**1.50**	N/A	**4.64**	0.60	1.30	N/A
Loopy-SLAM [23]	✗	3.79	**3.38**	7.03	1.62	3.41	3.85
3D Gaussian Splatting							
SplaTAM [16]	✗	3.35	6.54	11.13	1.24	5.16	5.48
MonoGS [24]	✗	1.59	7.03	8.55	1.44	1.49	4.02
Gaussian-SLAM [51]	✗	2.73	6.03	14.92	1.39	5.31	6.08
*Photo-SLAM [12]	✗	2.60	N/A	N/A	**0.35**	**1.00**	N/A
LoopSplat [54]	✗	2.08	3.54	6.24	1.58	3.22	3.33
CG-SLAM [11]	✓	2.43	4.54	9.39	1.20	2.45	4.0
VarSplat (Ours)	✓	1.80	3.40	6.05	1.41	3.36	**3.20**

Reconstruction. As shown in Table 5, Depth L1 decreases slightly (0.50 vs. 0.51 for LoopSplat) and F1 is essentially unchanged (90.2 vs. 90.4), indicating tighter alignment without surface inflation. As noted in LoopSplat [54],

Table 4. Tracking Performance on ScanNet. UC indicates uncertainty. VarSplat achieves the best overall, showing robustness to noisy indoor scenes.

Method	UC	00	59	106	169	181	207	Avg.
Neural Implicit Fields								
Co-SLAM [41]	✗	7.1	11.1	9.4	**5.9**	11.8	7.1	8.7
NICE-SLAM [55]	✗	12.0	14.0	7.9	10.9	13.4	6.2	10.7
ESLAM [15]	✗	7.3	8.5	7.5	6.5	9.0	5.7	7.4
Point-SLAM [30]	✗	10.2	10.8	8.7	7.2	22.2	14.8	12.3
GO-SLAM [52]	✗	5.4	7.5	7.0	7.0	**6.8**	6.9	6.8
Loopy-SLAM [23]	✗	**4.2**	7.5	8.3	7.5	10.6	7.9	7.7
Uni-SLAM [42]	✓	6.1	7.8	7.4	5.8	9.8	5.2	7.0
3D Gaussian Splatting								
MonoGS [24]	✗	9.8	32.1	8.9	10.7	21.8	7.9	15.2
SplaTAM [16]	✗	10.1	17.7	11.7	7.5	5.6	7.5	10.0
Gaussian-SLAM [51]	✗	21.2	12.8	13.5	13.6	21.0	13.4	15.9
LoopSplat [54]	✗	6.2	7.1	7.4	10.6	8.5	6.6	7.7
CG-SLAM [11]	✓	7.1	7.5	8.9	8.2	11.6	**5.3**	8.1
VarSplat (Ours)	✓	4.9	**5.8**	6.7	6.7	8.9	6.2	**6.5**

Table 5. Reconstruction Performance on Replica. VarSplat achieve third-best result after Loopy-SLAM and LoopSplat, showing that variance regularization preserves mesh quality.

Method	Metric	R0	R1	R2	Off0	Off1	Off2	Off3	Off4	Avg.
Neural Implicit Fields										
NICE-SLAM [55]	Depth L1	1.81	1.44	2.04	1.39	1.76	8.33	4.99	2.01	2.97
	F1 [%]	45.0	44.8	43.6	50.0	51.9	39.2	39.9	36.5	43.9
ESLAM [15]	Depth L1	0.97	1.07	1.28	0.86	1.26	1.71	1.43	1.06	1.18
	F1 [%]	81.0	82.2	83.9	78.4	75.5	77.1	75.9	79.1	79.1
Point-SLAM [30]	Depth L1	0.53	0.22	0.46	0.30	0.57	**0.49**	0.51	0.64	0.46
	F1 [%]	86.9	92.3	90.8	93.8	**91.6**	**89.0**	88.2	85.6	89.8
Loopy-SLAM [23]	Depth L1	**0.30**	**0.20**	**0.42**	**0.23**	**0.46**	0.60	**0.37**	**0.24**	**0.35**
	F1 [%]	**91.6**	**92.4**	90.6	**93.9**	**91.6**	88.5	**89.0**	**88.7**	**90.8**
3D Gaussian Splatting										
SplaTAM [16]	Depth L1	0.43	0.38	0.54	0.44	0.66	1.05	1.60	0.68	0.72
	F1 [%]	89.3	88.2	88.0	91.7	90.0	85.1	77.1	80.1	86.2
Gaussian-SLAM [51]	Depth L1	0.61	0.25	0.54	0.50	0.52	0.98	1.63	0.42	0.68
	F1 [%]	88.8	91.4	90.5	91.0	87.3	84.2	87.4	88.9	88.7
LoopSplat [54]	Depth L1	0.39	0.23	0.52	0.32	0.51	0.63	1.09	0.40	0.51
	F1 [%]	90.6	91.9	**91.1**	93.3	90.4	88.9	88.7	88.3	90.4
VarSplat (Ours)	Depth L1	0.39	0.26	0.51	0.33	**0.46**	0.60	1.08	0.35	0.50
	F1 [%]	90.7	91.8	91.0	93.3	90.1	88.7	87.7	88.3	90.2

Table 6. Rendering performance on 3 datasets. VarSplat achieves competitive results on both synthetic and real-world datasets. Gray indicates evaluation on submaps rather than global map.

Dataset	Replica [36]			TUM [38]			ScanNet [5]		
Method	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓
NICE-SLAM [55]	24.42	0.892	0.233	14.86	0.614	0.441	17.54	0.621	0.548
ESLAM [15]	28.06	0.923	0.245	15.26	0.478	0.569	15.29	0.658	0.488
Point-SLAM [30]	35.17	0.975	0.124	16.62	0.696	0.526	19.82	0.751	0.514
Loopy-SLAM [23]	35.47	0.981	0.109	12.94	0.489	0.645	15.23	0.629	0.671
SplaTAM [16]	34.11	0.970	**0.100**	22.80	**0.893**	**0.178**	19.14	0.716	**0.358**
Gaussian-SLAM [51]	42.08	0.996	0.018	25.05	0.929	0.168	27.67	0.923	0.248
LoopSplat [54]	36.63	0.985	0.112	22.72	0.873	0.259	**24.92**	0.845	0.425
VarSplat (Ours)	**37.15**	**0.986**	0.109	**23.14**	0.883	0.248	**24.92**	**0.848**	0.422

Loopy-SLAM [23] and Point-SLAM [30] sample points using ground truth depth, which assumes perfect depth input. These results also show that using the per-pixel uncertainty map to regularize the photometric loss does not degrade mesh reconstruction quality.

Rendering. In Table 6, we evaluate rendering quality on input views for Replica, TUM-RGBD, and ScanNet, and in Table 7 we report novel view synthesis on ScanNet++. VarSplat consistently achieves competitive or superior rendering across all four datasets.

Table 7. Novel View Synthesis on ScanNet++ (PSNR ↑). VarSplat achives the best result.

Method	a	b	c	d	e	Avg.
ESLAM [15]	13.63	11.86	11.83	10.59	10.64	11.71
SplaTAM [16]	23.95	22.66	13.95	8.47	20.06	17.82
Gaussian-SLAM [51]	26.66	24.42	15.01	18.35	21.91	21.27
LoopSplat [54]	25.60	23.65	**15.87**	**18.86**	**22.51**	21.30
VarSplat (Ours)	**26.97**	**24.52**	15.19	18.07	21.92	**21.33**

Table 8. Effect of uncertainty on pose estimation.

Model	Tracking	Loop	Registration	ATE RMSE ↓
I	✗	✗	✗	8.20
II	✓	✗	✗	7.63
III	✓	✓	✗	7.49
IV	✗	✓	✓	7.51
V	✓	✓	✓	**6.53**

Table 9. Ablation on Variance Training.

Model	Track Frozen	Depth	Square L2	ATE RMSE ↓
I	✗	✓	✓	7.55
II	✓	✗	✓	7.17
II	✓	✓	✗	7.38
IV	✓	✓	✓	**6.53**

Table 10. Runtime on Replica/Room0 using A100 80GB. Per-frame runtime is computed as total optimization time divided by the sequence length.

Method	Mapping /fr(s)	Mapping /iter(ms)	Tracking /fr(s)	Tracking /iter(ms)	ATE RMSE
NICE-SLAM [55]	5.8	90.6	8.1	**20.8**	0.97
Point-SLAM [30]	28.7	93.1	7.1	29.0	0.61
SplaTAM [16]	5.8	96.8	3.5	86.8	0.31
LoopSplat [54]	**1.2**	**30.1**	**1.8**	29.3	0.28
VarSplat (Ours)	1.9	50.3	2.0	34.1	**0.20**

4.3. Ablation studies

We conduct all ablation studies on six ScanNet scenes.

Uncertainty on pose estimation. Table 8 and Figure 3 present the ablation where uncertainty is applied to tracking, loop detection, and registration. Without it, pose estimation on noisy regions can cause jitter and long-range drifts. Adding uncertainty to tracking smooths local motion, in loop detection reduces false closures on repeated structure, and in registration removes overlap ghosting and tightens medium range alignment. Using all three together gives the best results, allowing VarSplat both local and global robustness and providing more reliable 3D maps.

Variance Training. Table 9 analyze learning per-splat variance with Gaussian negative log-likelihood. Freezing variance during tracking avoids conflicts with pose optimization and leads to more stable trajectories. Moreover, loop closure happens after submaps construction to check overlap.

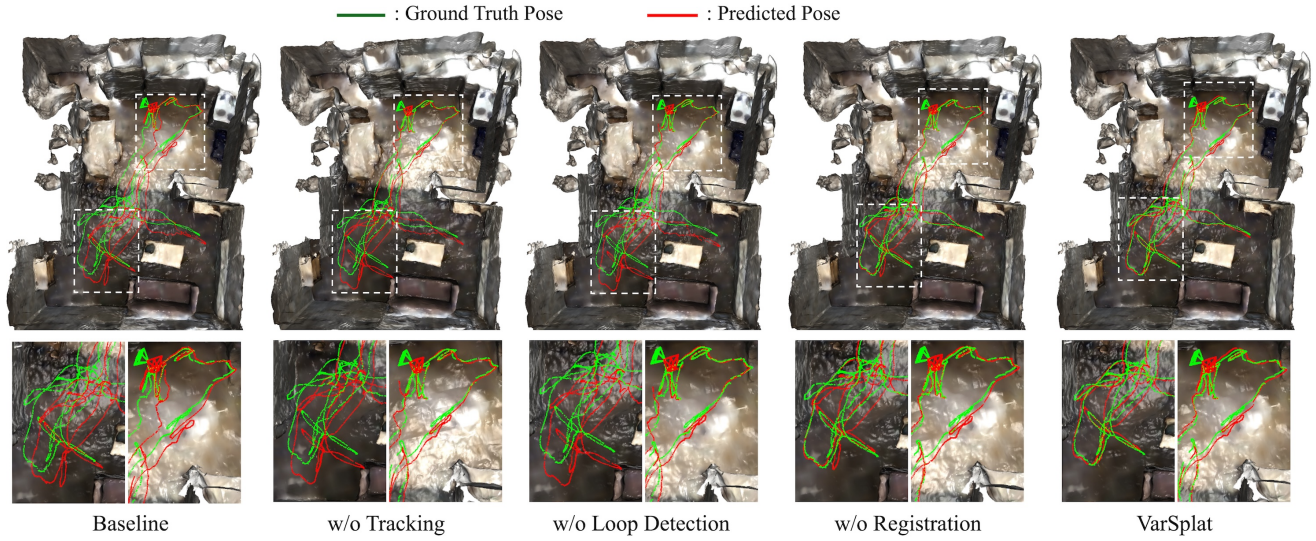

Figure 3. Uncertainty ablation on ScanNet (scene0181). Without uncertainty, tracking jitters, loop detection has long-range drift, and registration ghosts submaps. With VarSplat enabled, the trajectory is smooth and overlaps align.

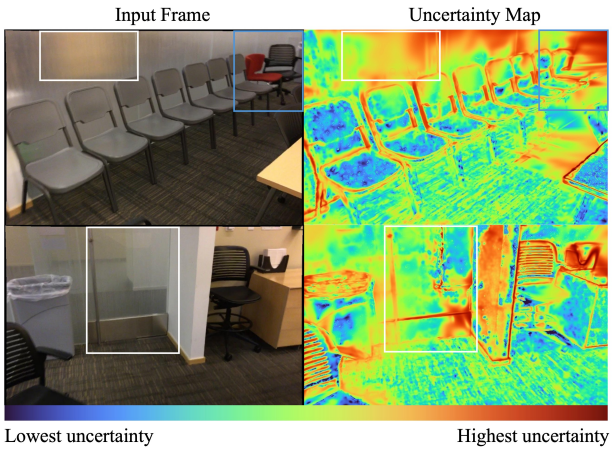

Figure 4. Visualization of challenging conditions (scene0169 ScanNet). In the top example, uncertainty is high on the texture-poor wall region (white box), while the map continues to grow (blue box). In the bottom example, the uncertainty map can distinguish reliable regions on transparent surface.

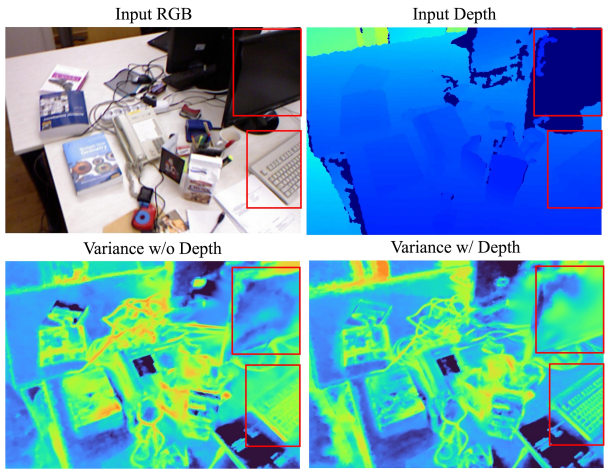

Figure 5. Per-pixel uncertainty with vs. without depth on TUM-RGBD (fr1/desk2). With depth, uncertainty focuses on textureless areas with depth holes and stays low on well constrained surfaces, avoiding overconfidence on glossy areas.

Therefore, gradients do not reach variance at this stage, so it remain frozen. Adding depth residuals tie uncertainty to geometry and prevents distractor-driven variance that misaligns submaps. Using square L2 for photometric and depth residuals matches the Gaussian model assumed by the NLL and provides smoother gradients than L1.

Runtime. As shown in Table 10, VarSplat achieve competitive runtime with recent 3DGS-SLAM systems. Learning and rendering variance raises computation cost, with corresponding improvements in tracking performance.

5. Conclusion

We present VarSplat, an uncertainty-aware 3DGS-SLAM system that learns per-splat appearance variance σ^2 and, via the law of total variance, renders differentiable per-pixel uncertainty map V. Both σ^2 and V are used for tracking, loop detection, and registration in submap-based pipeline, reducing drift and stabilizing poses. Across four datasets, this integration achieves robust and competitive-to-superior performance. Limitations and future works are provided in Supplementary Material.

Acknowledgement. This project was supported by resources provided by the Office of Research Computing at George Mason University (URL: https://orc.gmu.edu) and funded in part by grants from the National Science Foundation (Award Number 2018631).

References

[1] Luca Savant Aira, Diego Valsesia, and Enrico Magli. Modeling uncertainty for gaussian splatting. *IEEE Transactions on Neural Networks and Learning Systems*, 2025. 2

[2] Relja Arandjelovic, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. NetVLAD: Cnn architecture for weakly supervised place recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5297–5307, 2016. 1

[3] Carlos Campos, Richard Elvira, Juan J Gómez Rodríguez, José MM Montiel, and Juan D Tardós. ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE transactions on robotics*, 37(6):1874–1890, 2021. 6

[4] Jan Czarnowski, Tristan Laidlow, Ronald Clark, and Andrew J Davison. DeepFactors: Real-time probabilistic dense monocular slam. *IEEE Robotics and Automation Letters*, 5 (2):721–728, 2020. 3

[5] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. ScanNet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5828–5839, 2017. 6, 7, 1, 2

[6] Angela Dai, Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Christian Theobalt. BundleFusion: Real-time globally consistent 3d reconstruction using on-the-fly surface reintegration. *ACM Transactions on Graphics (ToG)*, 36(4): 1, 2017. 2

[7] Saptarshi Dasgupta, Akshat Gupta, Shreshth Tuli, and Rohan Paul. Actnerf: Uncertainty-aware active learning of nerf-based object models for robot manipulators using visual and re-orientation actions. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 13062–13069. IEEE, 2024. 2, 4

[8] Christian Forster, Zichao Zhang, Michael Gassner, Manuel Werlberger, and Davide Scaramuzza. SVO: Semidirect visual odometry for monocular and multicamera systems. *IEEE Transactions on Robotics*, 33(2):249–265, 2016. 3

[9] Lily Goli, Cody Reading, Silvia Sellán, Alec Jacobson, and Andrea Tagliasacchi. Bayes' rays: Uncertainty quantification for neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20061–20070, 2024. 2

[10] Alex Hanson, Allen Tu, Vasu Singla, Mayuka Jayawardhana, Matthias Zwicker, and Tom Goldstein. PUP 3D-GS: Principled uncertainty pruning for 3d gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5949–5958, 2025. 2

[11] Jiarui Hu, Xianhao Chen, Boyin Feng, Guanglin Li, Liangjing Yang, Hujun Bao, Guofeng Zhang, and Zhaopeng Cui. CG-SLAM: Efficient dense rgb-d slam in a consistent uncertainty-aware 3d gaussian field. In *European Conference on Computer Vision*, pages 93–112. Springer, 2024. 2, 3, 6, 7

[12] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21584–21593, 2024. 6

[13] Jiahui Huang, Shi-Sheng Huang, Haoxuan Song, and Shi-Min Hu. DI-Fusion: Online implicit 3d reconstruction with deep priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8932–8941, 2021. 2

[14] Wen Jiang, Boshu Lei, and Kostas Daniilidis. FisherRF: Active view selection and uncertainty quantification for radiance fields using fisher information. *arXiv preprint arXiv:2311.17874*, 2023. 2

[15] Mohammad Mahdi Johari, Camilla Carta, and François Fleuret. ESLAM: Efficient dense slam system based on hybrid representation of signed distance fields. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17408–17419, 2023. 2, 6, 7, 3

[16] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat track & map 3d gaussians for dense rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21357–21366, 2024. 2, 6, 7, 3

[17] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 2, 3, 4, 5, 1

[18] Christian Kerl, Jürgen Sturm, and Daniel Cremers. Dense visual slam for rgb-d cameras. In *2013 IEEE/RSJ international conference on intelligent robots and systems*, pages 2100–2106. IEEE, 2013. 1

[19] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems*, 37:80965–80986, 2024. 2

[20] Jonas Kulhanek, Songyou Peng, Zuzana Kukelova, Marc Pollefeys, and Torsten Sattler. WildGaussians: 3d gaussian splatting in the wild. *arXiv preprint arXiv:2407.08447*, 2024. 3, 5

[21] Sibaek Lee, Kyeongsu Kang, Seongbo Ha, and Hyeonwoo Yu. Bayesian NeRF: Quantifying uncertainty with volume density for neural implicit fields. *IEEE Robotics and Automation Letters*, 2025. 2

[22] Ruiqi Li and Yiu-ming Cheung. Variational multi-scale representation for estimating uncertainty in 3d gaussian splatting. *Advances in Neural Information Processing Systems*, 37:87934–87958, 2024. 2

[23] Lorenzo Liso, Erik Sandström, Vladimir Yugay, Luc Van Gool, and Martin R Oswald. Loopy-SLAM: Dense neural slam with loop closures. In *Proceedings of the IEEE/CVF*

conference on computer vision and pattern recognition, pages 20363–20373, 2024. 2, 3, 6, 7

[24] Hidenobu Matsuki, Riku Murai, Paul HJ Kelly, and Andrew J Davison. Gaussian Splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18039–18048, 2024. 2, 6, 7

[25] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 2

[26] Richard A Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-time dense surface mapping and tracking. In *2011 10th IEEE international symposium on mixed and augmented reality*, pages 127–136. Ieee, 2011. 1, 2

[27] Richard A Newcombe, Steven J Lovegrove, and Andrew J Davison. DTAM: Dense tracking and mapping in real-time. In *2011 international conference on computer vision*, pages 2320–2327. IEEE, 2011. 1, 2

[28] Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Colored point cloud registration revisited. In *Proceedings of the IEEE international conference on computer vision*, pages 143–152, 2017. 1

[29] Matia Pizzoli, Christian Forster, and Davide Scaramuzza. REMODE: Probabilistic, monocular dense reconstruction in real time. In *2014 IEEE international conference on robotics and automation (ICRA)*, pages 2609–2616. IEEE, 2014. 2

[30] Erik Sandström, Yue Li, Luc Van Gool, and Martin R Oswald. Point-SLAM dense neural point cloud-based slam. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18433–18444, 2023. 2, 6, 7, 3

[31] Erik Sandström, Kevin Ta, Luc Van Gool, and Martin R Oswald. UncLe-SLAM: Uncertainty learning for dense neural slam. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4537–4548, 2023. 2, 3

[32] Paul-Edouard Sarlin, Cesar Cadena, Roland Siegwart, and Marcin Dymczyk. From coarse to fine: Robust hierarchical localization at large scale. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12716–12725, 2019. 1

[33] Thomas Schops, Torsten Sattler, and Marc Pollefeys. Bad SLAM: Bundle adjusted direct rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 134–144, 2019. 2

[34] Jianxiong Shen, Adria Ruiz, Antonio Agudo, and Francesc Moreno-Noguer. Stochastic neural radiance fields: Quantifying uncertainty in implicit 3d representations. In *2021 International Conference on 3D Vision (3DV)*, pages 972–981. IEEE, 2021. 2

[35] Jianxiong Shen, Antonio Agudo, Francesc Moreno-Noguer, and Adria Ruiz. Conditional-flow NeRF: Accurate 3d modelling with reliable uncertainty quantification. In *European Conference on Computer Vision*, pages 540–557. Springer, 2022. 2

[36] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019. 6, 7, 1, 2

[37] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012. 6

[38] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012. 6, 7, 1, 2

[39] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J Davison. imap: Implicit mapping and positioning in real-time. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6229–6238, 2021. 2

[40] Yijie Tang, Jiazhao Zhang, Zhinan Yu, He Wang, and Kai Xu. MIPS-fusion: Multi-implicit-submaps for scalable and robust online neural rgb-d reconstruction. *ACM Transactions on Graphics (TOG)*, 42(6):1–16, 2023. 6

[41] Hengyi Wang, Jingwen Wang, and Lourdes Agapito. Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13293–13302, 2023. 2, 6, 7

[42] Shaoxiang Wang, Yaxu Xie, Chun-Peng Chang, Christen Millerdurai, Alain Pagani, and Didier Stricker. Uni-SLAM: Uncertainty-aware neural implicit slam for real-time dense indoor scene reconstruction. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2228–2239. IEEE, 2025. 2, 3, 6, 7

[43] Thomas Whelan, Michael Kaess, Hordur Johannsson, Maurice Fallon, John J Leonard, and John McDonald. Real-time large-scale dense rgb-d slam with volumetric fusion. *The International Journal of Robotics Research*, 34(4-5):598–626, 2015. 2

[44] Thomas Whelan, Stefan Leutenegger, Renato F Salas-Moreno, Ben Glocker, and Andrew J Davison. ElasticFusion: Dense slam without a pose graph. In *Robotics: science and systems*. Rome, 2015. 1, 2

[45] Joey Wilson, Marcelino Almeida, Min Sun, Sachit Mahajan, Maani Ghaffari, Parker Ewen, Omid Ghasemalizadeh, Cheng-Hao Kuo, and Arnie Sen. Modeling uncertainty in 3d gaussian splatting through continuous semantic splatting. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3284–3290. IEEE, 2025. 2

[46] Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-NeRF: Point-based neural radiance fields. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5438–5448, 2022. 2

[47] Ziheng Xu, Qingfeng Li, Chen Chen, Xuefeng Liu, and Jianwei Niu. Glc-SLAM: Gaussian splatting slam with efficient loop closure. *arXiv preprint arXiv:2409.10982*, 2024. 3

[48] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. GS-SLAM: Dense visual slam with

3d gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19595–19604, 2024. 2, 4, 6

[49] Xingrui Yang, Hai Li, Hongjia Zhai, Yuhang Ming, Yuqian Liu, and Guofeng Zhang. Vox-Fusion: Dense tracking and mapping with voxel-based neural implicit representation. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 499–507. IEEE, 2022. 2

[50] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. ScanNet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12–22, 2023. 6, 1, 2

[51] Vladimir Yugay, Yue Li, Theo Gevers, and Martin R Oswald. Gaussian-SLAM: Photo-realistic dense slam with gaussian splatting. *arXiv preprint arXiv:2312.10070*, 2023. 2, 3, 4, 5, 6, 7, 1

[52] Youmin Zhang, Fabio Tosi, Stefano Mattoccia, and Matteo Poggi. Go-SLAM: Global optimization for consistent 3d instant reconstruction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3727–3737, 2023. 2, 6, 7

[53] Jianhao Zheng, Zihan Zhu, Valentin Bieri, Marc Pollefeys, Songyou Peng, and Iro Armeni. WildGS-SLAM: Monocular gaussian splatting slam in dynamic environments. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 11461–11471, 2025. 2, 3, 5

[54] Liyuan Zhu, Yue Li, Erik Sandström, Shengyu Huang, Konrad Schindler, and Iro Armeni. LoopSplat: Loop closure by registering 3d gaussian splats. In *2025 International Conference on 3D Vision (3DV)*, pages 156–167. IEEE, 2025. 2, 3, 4, 5, 6, 7, 1

[55] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R Oswald, and Marc Pollefeys. Nice-SLAM: Neural implicit scalable encoding for slam. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12786–12796, 2022. 2, 6, 7, 3

[56] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Surface splatting. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pages 371–378, 2001. 3

VarSplat: Uncertainty-aware 3D Gaussian Splatting for Robust RGB-D SLAM

Supplementary Material

In Supplementary Material, we provide implementation details with hyperparameters and per-scene results that support the conclusions in the main paper.

- **Implementation.** System hardware and software details, along with changes to the rasterizer to render appearance variance.
- **Additional results.** Per scene PSNR, SSIM, and LPIPS on Replica, TUM RGB-D, and ScanNet
- **Limitations and future work.** Discussion of current limitations and directions for applying appearance variance in dynamic scenarios.

S1. Implementation.

System Details. We implement VarSplat in Python 3.10 and PyTorch 2.4.1 on NVIDIA A100 80 GB GPU with CUDA 12.6. Starting from the original 3DGS rasterizer [17] and its depth-rendering extension with pose [51], we extend the renderer to propagate per-splat appearance variance via the law of total variance (Eq. 9 in the main paper) to obtain a differentiable per-pixel uncertainty map.

Hyperparameters. Defaults follow LoopSplat [54] for fair comparison. We report the settings used in our experiments in Table S1, including λ_c for tracking loss, learning rate l_r for rotation and l_t for translation, and optimization iterations iter_t and iter_m for tracking and mapping processes, τ for variance-based weighting. Specifically, τ controls the sharpness of the uncertainty-aware weighting function. For scenes where uncertainty-based regularization is less critical or requires minimal calibration, we set $\tau = 100$ to effectively maintain nearly uniform weight distribution across pixels and splats, thereby softening the penalty on high-variance regions. Unless noted, $\lambda_{\text{color}}, \lambda_{\text{depth}}, \lambda_{\text{reg}}$ are set to 1 and λ_{var} is set to 0.0001 in mapping loss $\mathcal{L}_{\text{map}}$ for all datasets.

Table S1. Per-Dataset Hyperparameters.

Params	Replica [36]	TUM-RGBD [38]	ScanNet [5]	ScanNet++ [50]
λ_c	0.95	0.6	0.6	0.5
l_r	0.0002	0.002	0.002	0.002
l_t	0.002	0.01	0.01	0.01
iter_t	60	200	200	300
iter_m	100	100	100	500
τ	10	50	5	10

Tracking loss. In the tracking loss, the inlier mask M_{inlier} filters pixels whose depth residual is extreme. Concretely, a pixel is removed if its depth error exceeds 50 times the median depth error in the current frame. Pixels without valid depth are also excluded from pose optimization. For soft alpha masking $M_{\text{alpha}} = \alpha^3$, we follow prior work [51, 54] for loss weighting. On ScanNet++, we reinitialize the current-frame pose with ICP odometry [28] whenever the tracking loss exceeds 50 times the running average.

Submap. Based on motion heristics, new submap is triggered with displacement threshold $d_{\text{thre}} = 0.5[m]$ and rotation threshold $\theta_{\text{thre}} = 50°$[54]. Due to motion blur and far camera poses in ScanNet and ScanNet++, we use a different approach for submap initialization as setting fixed iterations of 50 and 100 frames. For new keyframe, we uniformly sample M_k points to meet alpha value condition or depth discrepancy condition. M_k is limited for per-dataset setting, 30k for TUM-RGBD and ScanNet, while 100K for ScaNet++, and all available points for Replica. The alpha threshold α_{thre} is set to 0.98 for all datasets. The depth discrepancy condition is depth error passes 40 times median depth error of current frame. New Gaussians are initialized with opacity 0.5 and initial scales to the nearest neighbor. After finishing mapping optimization, we prune Gaussians based on fixed opacity threshold with 0.1 for Replica and 0.5 for remaining datasets.

Loop Detection. We use NetVLAD [2] with the VGG16-NetVLAD-Pitts30K weights from HLoc [32], similar to [54]. For each submap, we compute cosine similarities among its keyframes and define a self-similarity score s^i_{self} as the p-th percentile of these values. We set $p = 50$ on Replica, TUM-RGBD, and ScanNet, and $p = 33$ on ScanNet++. After obtaining submap-to-submap similarities, we rescore them with the submap-level reliability ratio $r^{(s)}$ derived from per-splat variances, so $\text{sim}_k = \text{cross\_sim}_k r^{(q)} r^{(k)}$. We then filter candidates by an overlap ratio computed with the front-end poses.

3DGS Registration For each accepted loop, we select the top $k = 2$ overlapping viewpoints using NetVLAD and solve a multi-view pose estimation between the two submaps [54]. We optimize the relative pose and per-view exposure coefficients for the selected viewpoints. The registration objective uses the photometric loss weighted by the per-pixel variance weight $\tilde{w_p}$ and an unweighted depth loss, as defined in the tracking and registration losses earlier. During registration, variance parameters are fixed.

Datasets. The DSLR sequences contain abrupt motions, so we evaluate only the first 250 frames of each sequence. Table S2 reports, for all evaluated scenes, the average frame to frame translation, rotation, and the frame count. Among the benchmarks, ScanNet++ shows about $10\times$ larger motion per frame than the others, which makes pose estimation more challenging and more prone to drift, highlighting the effectiveness of our approach in reducing drift for real-world SLAM.

Table S2. Frame Motion on Replica [36], TUM-RGBD [38], ScanNet [5], and ScanNet++ [50].

Dataset	Replica	TUM	ScanNet	ScanNet++
Translation (cm)	1.07	1.39	1.34	14.77
Rotation (°)	0.50	1.37	0.69	13.43

S2. Additional Results.

In the main paper we report dataset level averages for rendering quality. Tables S3 to S5 list per-scene results for Replica, TUM RGB-D, and ScanNet. Across the three datasets, VarSplat is competitive on most scenes.

S3. Limitations and Future Work

Although VarSplat improves robustness, a depth-based approach for where and when to add Gaussians limits performance when depth is sparse or missing. As seen in the per-scene TUM-RGBD results, future work should explore joint learning of appearance and geometric uncertainty with depth completion. Additionally, learning and rendering variance add computation and memory. Future work includes variance sharing across splats, pruning, and lightweight approximations of the uncertainty map. Fianlly, Our experiments focus on mostly static scenes. Extending appearance variance to handle moving objects with variance-guided motion segmentation and dynamic mapping is a promising direction.

Table S3. Rendering Performance on Replica. * denotes evaluating on submaps instead of a global one.

Method	Metric	Rm0	Rm1	Rm2	Off0	Off1	Off2	Off3	Off4	Avg.
NICE-SLAM [55]	PSNR↑	22.12	22.47	24.52	29.07	30.34	19.66	22.23	24.94	24.42
	SSIM↑	0.689	0.757	0.814	0.874	0.886	0.797	0.801	0.856	0.809
	LPIPS↓	0.330	0.271	0.208	0.229	0.181	0.235	0.209	0.198	0.233
ESLAM [15]	PSNR↑	25.25	27.39	28.09	30.33	27.04	27.99	29.27	29.15	28.06
	SSIM↑	0.874	0.890	0.935	0.934	0.910	0.942	0.953	0.948	0.923
	LPIPS↓	0.315	0.296	0.245	0.213	0.254	0.238	0.186	0.210	0.245
Point-SLAM [30]	PSNR↑	32.40	34.08	35.50	38.26	39.16	33.99	33.48	33.49	35.17
	SSIM↑	0.974	0.977	0.982	0.983	0.986	0.960	0.960	0.979	0.975
	LPIPS↓	0.113	0.116	0.111	0.100	0.118	0.156	0.132	0.142	0.124
SplaTAM [16]	PSNR↑	32.86	33.89	35.25	38.26	39.17	31.97	29.70	31.81	34.11
	SSIM↑	0.980	0.970	0.980	0.980	0.980	0.970	0.950	0.950	0.970
	LPIPS↓	0.070	0.100	0.080	0.090	0.090	0.100	0.120	0.150	0.100
* Gaussian-SLAM [51]	PSNR↑	38.88	41.80	42.44	46.40	45.29	40.10	39.06	42.65	42.08
	SSIM↑	0.993	0.996	0.996	0.998	0.997	0.997	0.997	0.997	0.996
	LPIPS↓	0.017	0.018	0.019	0.015	0.016	0.020	0.020	0.020	0.018
LoopSplat [54]	PSNR↑	33.07	35.32	36.16	40.82	40.21	34.67	35.67	37.10	36.63
	SSIM↑	0.973	0.978	0.985	0.992	0.990	0.985	0.990	0.989	0.985
	LPIPS↓	0.116	0.122	0.111	0.085	0.123	0.140	0.096	0.106	0.112
VarSplat	PSNR↑	**33.93**	**35.82**	**36.50**	**41.06**	**41.12**	**35.61**	**36.05**	**37.49**	**37.16**
	SSIM↑	0.978	**0.981**	**0.986**	**0.992**	**0.991**	**0.986**	**0.990**	**0.990**	**0.986**
	LPIPS↓	0.105	0.117	0.109	**0.082**	0.120	0.137	**0.093**	**0.100**	0.109

Table S4. Rendering Performance on TUM RGB-D. * denotes evaluating on submaps instead of a global one.

Method	Metric	fr1/desk	fr2/xyz	fr3/office	Avg.
NICE-SLAM [55]	PSNR↑	13.83	17.87	12.89	14.86
	SSIM↑	0.569	0.718	0.554	0.614
	LPIPS↓	0.482	0.344	0.498	0.441
ESLAM [15]	PSNR↑	11.29	17.46	17.02	15.26
	SSIM↑	0.666	0.310	0.457	0.478
	LPIPS↓	0.358	0.698	0.652	0.569
Point-SLAM [30]	PSNR↑	13.87	17.56	18.43	16.62
	SSIM↑	0.627	0.708	0.754	0.696
	LPIPS↓	0.544	0.585	0.448	0.526
SplaTAM [16]	PSNR↑	22.00	24.50	21.90	22.80
	SSIM↑	0.857	0.947	0.876	0.893
	LPIPS↓	0.232	0.100	0.202	0.178
* Gaussian-SLAM [51]	PSNR↑	24.01	25.02	26.13	25.05
	SSIM↑	0.924	0.924	0.939	0.929
	LPIPS↓	0.178	0.186	0.141	0.168
LoopSplat [54]	PSNR↑	22.03	22.68	23.47	22.72
	SSIM↑	0.849	0.892	0.879	0.873
	LPIPS↓	0.307	0.217	0.253	0.259
VarSplat	PSNR↑	**22.03**	23.85	**23.53**	**23.14**
	SSIM↑	0.847	0.920	**0.882**	0.883
	LPIPS↓	0.311	0.189	0.248	0.248

Table S5. Rendering Performance on ScanNet. * denotes evaluating on submaps instead of a global one.

Method	Metric	0000	0059	0106	0169	0181	0207	Avg.
NICE-SLAM [55]	PSNR↑	18.71	16.55	17.29	18.75	15.56	18.38	17.54
	SSIM↑	0.641	0.605	0.646	0.629	0.562	0.646	0.621
	LPIPS↓	0.561	0.534	0.510	0.534	0.602	0.552	0.548
ESLAM [15]	PSNR↑	15.70	14.48	15.44	14.56	14.22	17.32	15.29
	SSIM↑	0.687	0.632	0.628	0.656	0.696	0.653	0.658
	LPIPS↓	0.449	0.450	0.529	0.486	0.482	0.534	0.488
Point-SLAM [30]	PSNR↑	21.30	19.48	16.80	18.53	18.23	20.56	19.16
	SSIM↑	0.806	0.765	0.676	0.688	0.823	0.750	0.751
	LPIPS↓	0.485	0.499	0.544	0.542	0.471	0.544	0.514
SplaTAM [16]	PSNR↑	19.33	19.27	17.73	21.97	16.76	19.80	19.14
	SSIM↑	0.660	0.792	0.690	0.776	0.683	0.696	0.716
	LPIPS↓	0.438	0.289	0.376	0.281	0.402	0.341	0.358
* Gaussian-SLAM [51]	PSNR↑	28.54	26.21	23.27	28.60	27.79	28.63	27.67
	SSIM↑	0.926	0.934	0.926	0.917	0.923	0.913	0.923
	LPIPS↓	0.271	0.211	0.217	0.226	0.277	0.288	0.248
LoopSplat [54]	PSNR↑	24.99	23.23	23.35	26.80	24.82	26.33	24.92
	SSIM↑	0.840	0.831	0.846	0.877	0.824	0.854	0.845
	LPIPS↓	0.450	0.400	0.409	0.346	0.514	0.430	0.425
VarSplat	PSNR↑	**25.12**	23.16	**23.52**	**26.82**	24.67	26.20	**24.92**
	SSIM↑	**0.850**	**0.834**	**0.852**	**0.879**	0.820	**0.854**	**0.848**
	LPIPS↓	0.444	0.399	0.404	0.340	0.518	0.425	0.422